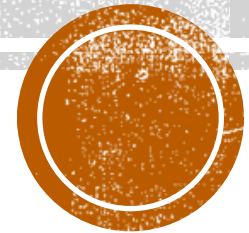




Module: Développer en Front-end

Comprendre les concepts de base du
web front-end



Filière: Développement digital – option web full stack

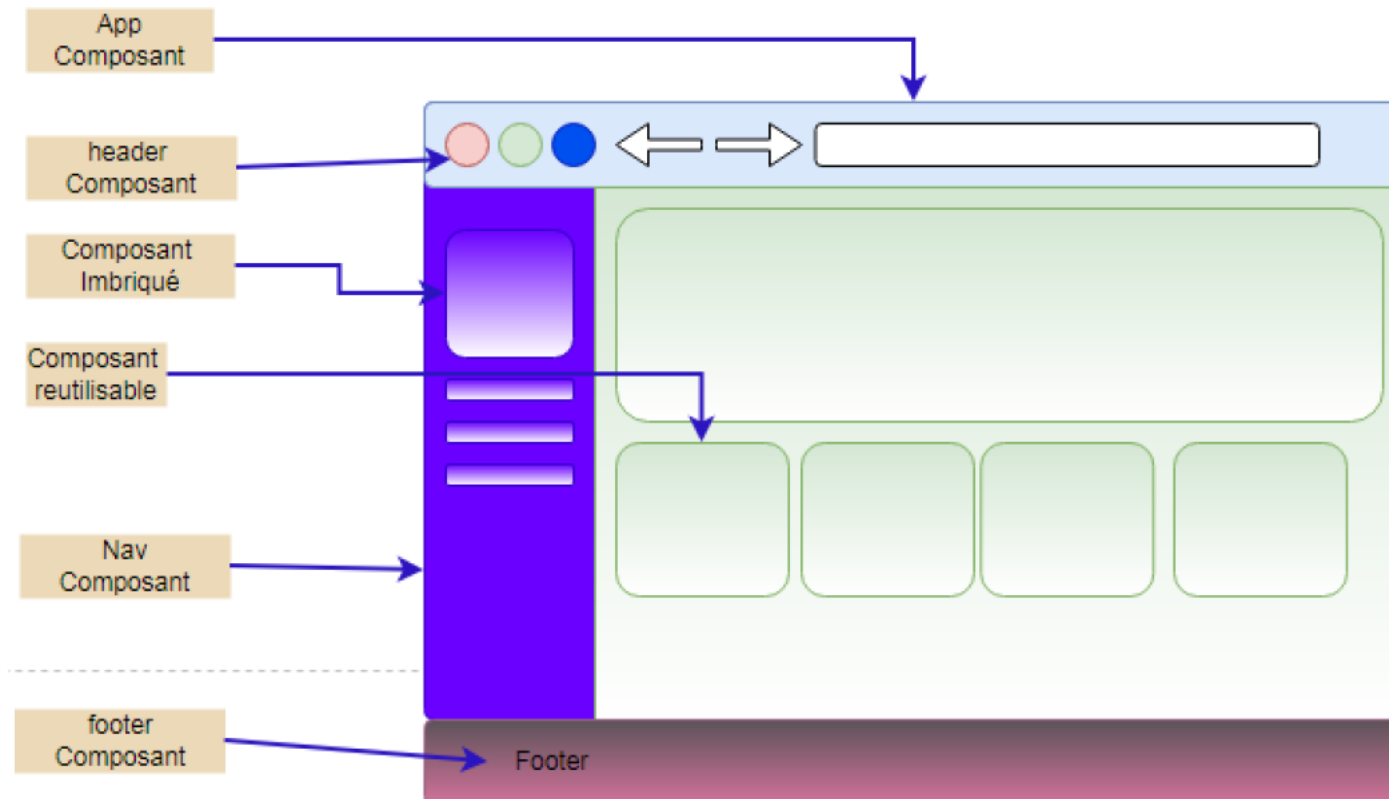
Réalisé par: ELMAUDNI

Différence entre front End et back end :

- Le front-end est la partie de l'application web avec laquelle l'utilisateur interagit directement. Il s'agit de l'interface utilisateur, qui comprend la conception, la disposition, les boutons, les formulaires, les graphiques, les animations, etc.
- Le front-end est construit à l'aide d'une combinaison de technologies telles que le langage de balisage hypertexte (HTML), JavaScript et les feuilles de style en cascade (CSS).
- Le back-end, également appelé côté serveur, se compose du serveur qui fournit les données à la demande, de l'application qui les canalise et de la base de données qui organise les informations.

Introduction à React

- React c'est une librairie java script pour créer des interfaces utilisateur frontend
- React c'est un utilitaire qui va nous aider à développer des interfaces utilisateur plus rapidement et facilement.
- React est une librairie développée par Facebook est qui toujours maintenu avec Facebook
- L'approche des bibliothèques et Framework Frontend consiste à découper l'interface utilisateur en un ensemble de morceaux dite composants réutilisable.

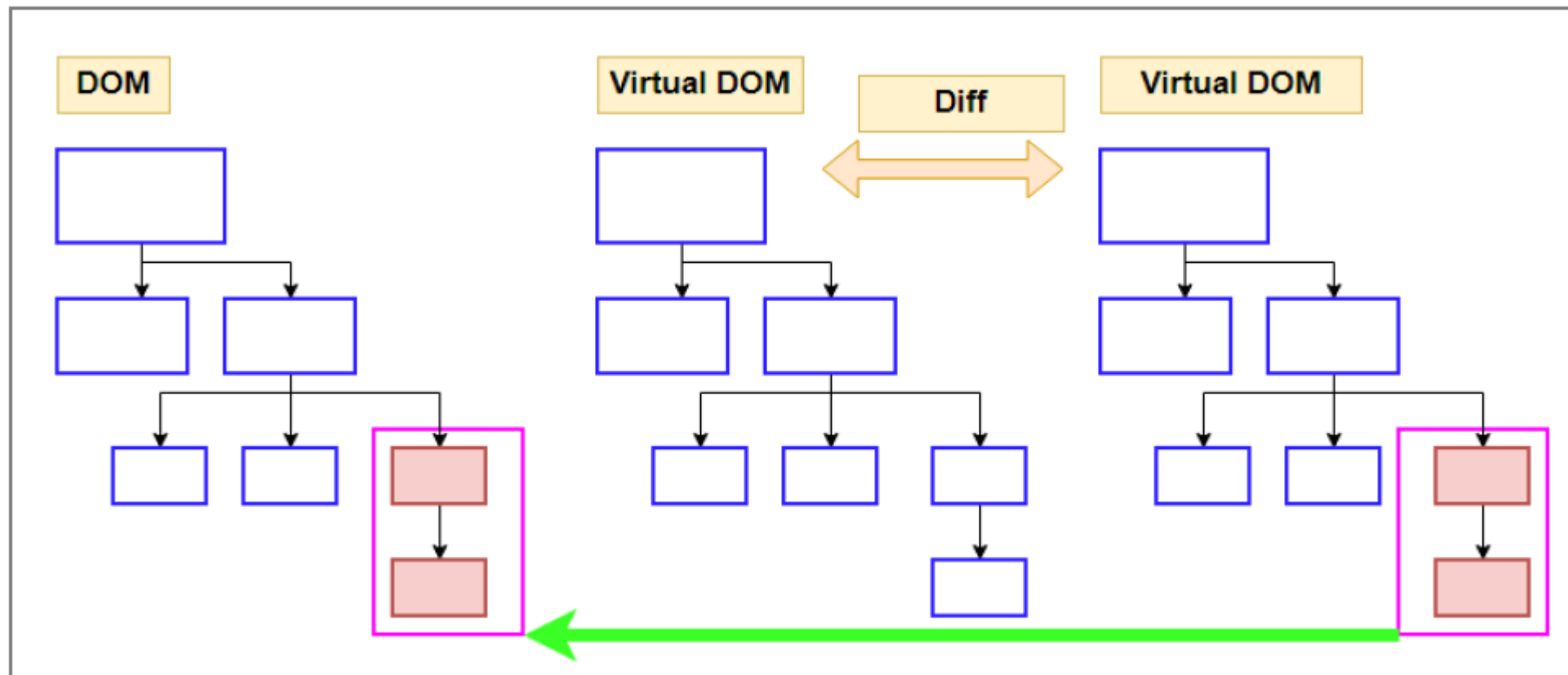


Avantages

- React est le framework open-source développé et créé par Facebook. Ce framework est le meilleur framework d'interface utilisateur de 2022, utilisé par la majorité des développeurs frontaux selon Framework frontal, React se distingue par son modèle d'objet de document (DOM) virtuel, qui présente ses excellentes fonctionnalités.
- **Avantages**
- Gain de temps lors de la réutilisation des composants
- Virtual DOM améliore à la fois l'expérience des utilisateurs et le travail du développeur
- Une bibliothèque open-source avec une diversité d'outils

Virtuel DOM :

- Le DOM virtuel (VDOM) est un concept de programmation dans lequel une représentation idéale ou "virtuelle" d'une interface utilisateur est conservée en mémoire et synchronisée avec le "vrai" DOM par une bibliothèque telle que ReactDOM. Ce processus s'appelle la réconciliation.



Les aspects avancés de JavaScript :

- Le développement Frontend avec React nécessite une bonne maitrise de java script ES6
- Voici quelques points précis concernant JavaScript (ECMAScript 2015) qui ont été introduits dans cette version: . .

Variables let et const

Template literals :

L'opérateur conditionnel ternaire

Classes

Destructuration :

Modules

Spread Operator (...)

Opérations sur Array: find, filtrer,map

Arrow Functions

Variables let et const

- let permet de déclarer des variables avec une portée de bloc, tandis que const déclare des variables constantes dont la valeur ne peut pas être réassignée.

```
let x = 10;
```

```
const pi = 3.14;
```

Les mots-clés **let** et **var** sont utilisés pour déclarer des variables en JavaScript, mais ils diffèrent principalement par leur portée et leur contexte d'exécution

Exemple

```
console.log("avant");
for(var i=0;i<2;i++){
  console.log(i);
}
console.log("après");
console.log(i);
Résultat:
avant
0
1
après
2
```

```
console.log("avant");
for(let i=0;i<2;i++){
  console.log(i);
}
console.log("après");
console.log(i);
Résultat:
```

avant

0

1

après

✖ ▶ Uncaught ReferenceError: i is not defined
at [script.js:6:13](#)

let : Les variables déclarées avec let ont une portée de bloc, ce qui signifie qu'elles sont limitées à l'intérieur du bloc de code dans lequel elles sont définies.

var : Les variables déclarées avec var ont une portée de fonction ou une portée globale, ce qui signifie qu'elles sont visibles dans toute la fonction dans laquelle elles sont déclarées

Template literals :

Exemple:

```
const salutation="salut " ;  
const nom="Rami" ;  
//avec template literal  
console.log(`${salutation} ${nom}`) ;  
//est identique a  
console.log(salutation+" "+nom);
```

Rendu

salut	Rami
salut	Rami

L'opérateur conditionnel ternaire

- L'opérateur conditionnel (ternaire) est un opérateur JavaScript qui prend trois opérandes : une condition suivie d'un point d'interrogation (?), puis une expression à exécuter si la condition est vraie suivie de deux-points (:), et enfin l'expression à exécuter si la condition est fausse. Cet opérateur est fréquemment utilisé comme alternative à une instruction if...else.
- L'opérateur conditionnel ternaire est très utilisé en react notamment en JSX exemple en Java Script ECS 6 :

```
let isMember=true;  
let remise=isMember==true?0.2:0.1  
console.log("remise: ",remise)
```

Résultat: 0.2

Classes

- ES6 a introduit une syntaxe de classe plus propre pour la création de classes et d'objets.

```
class Etudiant{  
  constructor(nom,age){  
    this.nom=nom;  
    this.age=age  
  }  
  info(){  
    return `Etudiant nom:${this.nom} a pour  
    age:${this.age}`  
  }  
}  
  
let et1= new Etudiant("Rami",23)  
console.log(et1.info());  
  
let et2= new Etudiant("Karimi",21)  
console.log(et2.info());
```

Rendu

```
Etudiant nom:Rami a pour age:23  
Etudiant nom:Karimi a pour age:21
```

Héritage de classe

```
class Stagiaire extends Etudiant{  
  constructor(nom,age,stage){  
    super(nom,age);  
    this.stage=stage; }  
  info(){return `Stagiaire nom:${this.nom} a pour age:${this.age} stage:${this.stage}`  
  }  
}  
//La méthode info() peut être codé de la manière suivante  
  
info(){  
  return `${super.info()} stage:${this.stage}`  
}
```

super(nom,age) :fait appelle au constructeur de la classe mère Etudiant

```
let stg1= new Stagiaire("Rami",23,"dev mobile");  
console.log(stg1.info());  
let stg2= new Stagiaire("Karimi",21,"dev web")  
console.log(stg2.info());
```

```
Stagiaire nom:Rami a pour age:23 stage:dev mobile  
Stagiaire nom:Karimi a pour age:21 stage:dev web
```

Modules

- Les modules ES6 permettent d'organiser le code en utilisant import et export, facilitant ainsi la gestion des dépendances.
- Un module c'est un fichier Java Script, qui contient les mots clés **export default** ou **export** elles les valeurs, classes, et fonctions qu'on veut exporter
- Exemple

NB: Dans un module on peut faire seulement un seul export default, alors qu'on peut faire plusieurs exports. Dans le module ci-dessous il un seul export default : export default Etudiant et deux export :

```
Module Etudiant.js
const Etablissement='ISGI'
class Etudiant{
  constructor(nom,age){
    this.nom=nom;

    this.age=age
  }
}
function info(etudiant){
  return `nom:${etudiant.nom}
age:${etudiant.age}`
}
export default Etudiant
export {Etablissement,info}
```

Identique à :

```
export const Etablissement='ISGI'
export default class Etudiant{
  constructor(nom,age){
    this.nom=nom;
    this.age=age
  }
}
export function info(etudiant){
  return `nom:${etudiant.nom}
age:${etudiant.age}`
}
```

Modules(Suite)

- L'élément exporté par default est importé sans accolades, tandis les éléments exportés par export sont importés avec des accolades
- Module Index.js

```
import Etudiant,{Etablissement,info} from './Etudiant.js'  
let etd1= new Etudiant("Rami",33)  
console.log(info(etd1))
```

- Utilisation des aléas :

```
import Etd,{Etablissement as Etab,info} from './Etudiant.js'  
let etd1= new Etd("Rami",33)  
console.log(info(etd1)+" Etablissement: "+Etab)
```

- NB: pour utiliser les modules dans la balise <script>

```
<html lang="en">  
<head><meta charset="UTF-8"></head>  
<body>  
<script src="Index.js" type="module"></script>  
</body>  
</html>
```

Destructuration

- La syntaxe d'affectation de déstructuration est une expression JavaScript qui permet de décompresser des valeurs de tableaux, ou des propriétés d'objets, dans des variables distinctes.
- Destructuration d'un objet :**

```
const personne={  
  nom:"fatihi",  
  age:23,  
  adresse:{rue:14,ville:"casa"}  
}  
  
const nom=personne.nom  
const age=personne.age  
const rue=personne.adresse.rue  
const ville=personne.adresse.ville  
console.log(nom,age,rue,ville)  
  
//est identique à:  
const {nom,age,adresse:{rue},adresse:{ville}}=personne  
console.log(nom,age,rue,ville)
```

Destructuration

- **Destructuration d'un tableau** : Supposons que vous ayez un tableau d'éléments et que vous vouliez extraire des éléments spécifiques du tableau dans des variables distinctes. Voici un exemple :

```
const numbers = [1, 2, 3, 4, 5];  
  
// Destructuration d'un tableau  
const [first, second, ...rest] = numbers;  
  
console.log(first); // 1  
console.log(second); // 2  
console.log(rest); // [3, 4, 5]
```

- Dans cet exemple, les deux premiers éléments du tableau sont extraits dans les variables first et second, tandis que le reste des éléments est stocké dans un tableau rest

Destructuring avec fonction

- La destructuration peut également être utilisée avec les fonctions en JavaScript pour extraire des valeurs d'objets passés en tant qu'arguments de fonction.
- Supposons que vous ayez une fonction qui prend un objet en tant qu'argument, vous pouvez extraire les propriétés de cet objet directement dans les paramètres de la fonction :

```
const personne={  
  nom:"fatihi",  
  age:23,  
  adresse:{rue:14,ville:"casa"}  
}  
function  
presentation({nom,age,adresse:{rue},adresse:{ville}}){  
  console.log("salut ",nom,age,rue,ville)  
}  
presentation(personne)
```

Opérations sur Array: La méthode map

- map est une méthode très populaire elle permet de :
 - Retourne un nouveau Array
 - Le nouveau Array a la même taille que l'Array d'origine
 - Utilise les valeurs de l'Array d'origine pour faire le nouveau Array

Exemple:

```
const personnes = [  
  { nom: "Rami", age: 33, estMember: true },  
  { nom: "Fatihi", age: 24, estMember: false },  
  { nom: "Chakib", age: 45, estMember: true },  
  { nom: "Mounir", age: 37, estMember: false },  
];  
const noms=personnes.map(function(pers){return  
pers.nom}) ;  
console.log(noms) ;
```

Le rendu : ▶ (4) ['Rami', 'Fatihi', 'Chakib', 'Mounir']

La méthode filter

- La méthode filter() crée une copie d'une partie d'un tableau donné, filtrée uniquement pour les éléments du tableau donné qui réussissent la condition implémentée par la fonction fournie.

Exemple:

```
const personnes = [
  { nom: "Rami", age: 33, estMember: true },
  { nom: "Fatihi", age: 24, estMember: false },
  { nom: "Chakib", age: 45, estMember: true },
  { nom: "Mounir", age: 37, estMember: false },
];
const
persAgés=personnes.filter(function(pers){return
pers.age>=35})
console.log(persAgés)
```

Le rendu :

```
▼ (2) [{...}, {...}] ⓘ
  ► 0: {nom: 'Chakib', age: 45, estMember: true}
  ► 1: {nom: 'Mounir', age: 37, estMember: false}
  length: 2
  ► [[Prototype]]: Array(0)
```

La méthode find

- La méthode find() renvoie le premier élément du tableau fourni qui satisfait la fonction de test fournie. Si aucune valeur ne satisfait la fonction de test, undefined est renvoyé.

```
const personnes = [  
  { nom: "Rami", age: 33, estMember: true },  
  { nom: "Fatihi", age: 24, estMember: false },  
  { nom: "Chakib", age: 45, estMember: true },  
  { nom: "Mounir", age: 37, estMember: false },  
];  
const pers1=personnes.find(function(pers){return  
pers.age>=45})  
console.log(pers1)  Le rendu :  
    ► {nom: 'Chakib', age: 45, estMember: true}
```

La méthode reduce

- La méthode reduce() exécute une fonction de rappel "reducer" fournie par l'utilisateur sur chaque élément du tableau, dans l'ordre, en transmettant la valeur de retour du calcul sur l'élément précédent. Le résultat final de l'exécution du réducteur sur tous les éléments du tableau est une valeur unique.

```
const clients = [
  { nom: "Rami", montant: 4500 },
  { nom: "Karimi", montant: 2300 },
  { nom: "Chaouki", montant: 5500 },
  { nom: "Ramoun", montant: 7700 },
];
const
totalMontants=clients.reduce(function(total,client
){
return total+=client.montant
},0);
console.log(totalMontants)
```

Arrow fonction

- Les arrow fonctions sont juste un moyen plus court et plus succinct de créer une fonction (sans nom). Les arrow fonctions peuvent être utilisées presque partout où une fonction classique peut être utilisée, sauf qu'elles ne peuvent pas être utilisées comme constructeurs

```
function somme(a,b){  
  return a+b  
}  
//Peut-être écrite en utilisant arrow fonction  
  
const mySomme=(a,b)=>a+b ;  
console.log(mySomme(3,4))
```

NB: Les fonctions fléchées sont généralement appelées fonctions anonymes en raison de leur absence de nom. Si vous avez besoin de faire référence à une fonction fléchée, vous devrez l'affecter à une variable ou à un attribut d'objet, comme nous l'avons fait ici ; sinon, vous ne pourrez pas l'utiliser.

Exemple:
`const fact3 = n => (n === 0 ? 1 : n * fact3(n - 1));`

Spread Operator (...)

- L'opérateur de propagation, également appelé "spread operator", est utilisé pour décomposer un tableau ou un objet en ses éléments individuels. Il est généralement utilisé dans le contexte de la création de nouvelles structures de données en combinant ou en copiant des éléments. Voici comment fonctionne le spread operator avec des exemples :

Spread Operator avec les tableaux :

- **Copier un tableau** : Le spread operator peut être utilisé pour créer une copie d'un tableau existant.

```
const originalArray = [1, 2, 3];  
const copiedArray = [...originalArray];  
console.log(copiedArray); // [1, 2, 3]
```

Concaténer des tableaux : Vous pouvez concaténer deux tableaux en utilisant le spread operator.

```
const array1 = [1, 2, 3];  
const array2 = [4, 5, 6];  
const combinedArray = [...array1, ...array2];  
console.log(combinedArray); // [1, 2, 3, 4, 5, 6]
```

Ajouter un élément à un tableau : Le spread operator peut également être utilisé pour ajouter un nouvel élément à un tableau existant.

```
const originalArray = [1, 2, 3];  
const newArray = [...originalArray, 4];  
console.log(newArray); // [1, 2, 3, 4]
```

Spread Operator avec les objets :

Copier un objet : Vous pouvez utiliser le spread operator pour créer une copie superficielle d'un objet existant.

```
const originalObject = { name: 'Alice', age: 30 };  
const copiedObject = { ...originalObject };  
console.log(copiedObject); // { name: 'Alice', age: 30 }
```

Combiner des objets : Vous pouvez combiner les propriétés de plusieurs objets en un seul en utilisant le spread operator.

```
const object1 = { name: 'Alice' };  
const object2 = { age: 30 };  
const combinedObject = { ...object1, ...object2 };  
console.log(combinedObject); // { name: 'Alice', age: 30 }
```

Modifier un objet : Le spread operator peut être utilisé pour créer une nouvelle version d'un objet en ajoutant ou en modifiant des propriétés.

```
const originalObject = { name: 'Alice', age: 30 };  
const updatedObject = { ...originalObject, age: 31 };  
console.log(updatedObject); // { name: 'Alice', age: 31 }
```